Multi-Horizon Time-Series Forecasting for Enterprise Analytics Balancing Simplicity, Real-World Impact, and Research Support

Abstract

In enterprise decision-making, it is becoming ever more important to have precise forecasts of various business metrics (e.g., demand, sales, inventory requirements), not only for a single horizon but for several time horizons ahead. The proposed system is an enterprise-specific time-series multi-horizon forecasting pipeline based on three major concerns: simplicity, practical value, and research validity. The forecast generation involves interpretable statistical baselines (ARIMA, Prophet), robust ML approaches (LightGBM) as well as optional state-of-the-art DL models naturally suitable for multi-horizon prediction (N-BEATS, Temporal Fusion Transformer). The entire pipeline includes data ingestion, feature engineering, models training, evaluation, and serving stages to provide accurate and at the same time understandable forecasts for business users. Model performances are measured not only in traditional metrics (MAPE, RMSE, pinball loss) but also in terms of business impact (cost reduction due to better forecasts). Deployment of the forecast generation system is done by means of API and Dashboard services; the system is monitored and retrained periodically to preserve its performance as the input data changes. Thus, the proposed approach is production-ready as it utilizes well-validated techniques rather than unnecessary complex architectures.

1. Title Breakdown

The project title encodes four distinct scope decisions. Each phrase below maps to a concrete implication for how the system should be designed.

Phrase	What It Means	Implication for Scope
Multi-horizon	Predicting several future time steps at once (e.g., 1, 7, and 30 days ahead) rather than one step ahead.	Requires models/architectures natively suited to multi-step output, not just recursive single-step forecasting.
Time-series forecasting	Core ML task of using historical sequential data to predict future values.	Standard pipeline: ingestion, preprocessing, modeling, evaluation.
Enterprise analytics	The application domain — sales, demand, revenue, inventory, resource planning.	Business-metric-driven data with seasonality, holidays, promotions, and multiple related series.
Simplicity, impact, research support	Explicit project philosophy: avoid over-engineering while remaining credible and usable.	Favor well-validated, interpretable methods over exotic state-of-the-art architectures unless justified.

In plain terms: Build a forecasting system that predicts several future values ahead, tailored to business metrics, easy to build and explain, genuinely useful for decisions, and grounded in credible forecasting research — not a needlessly complex research showcase.

2. Technology Stack

Layer	Recommended Tools	Why
Language	Python	De facto standard for time-series and ML work.
Data handling	Pandas, NumPy, Polars (for large data)	Cleaning, resampling, feature engineering.
Classical / statistical models	statsmodels, pmdarima (ARIMA/SARIMA), Prophet	Strong, interpretable baselines — supports simplicity and real-world impact.
ML models	LightGBM, XGBoost with lag/window features	Fast, robust, easy to explain to business stakeholders.
Deep learning (optional)	PyTorch Forecasting, Darts, NeuralProphet, N-BEATS/TFT	Adds multi-horizon-native architectures without building from scratch.
Experiment tracking	MLflow or Weights & Biases	Tracks model versions, metrics, and reproducibility.
Orchestration	Prefect or Airflow	Automates retraining and scheduling in production.
Serving / API	FastAPI	Lightweight way to expose forecasts to enterprise systems.
Visualization / dashboard	Plotly, Streamlit, or Power BI	Makes forecasts consumable by non-technical stakeholders.
Storage	PostgreSQL, Parquet, or a data warehouse	Plugs into existing enterprise data infrastructure.

Guiding principle: Use Darts as a unifying library — it wraps ARIMA, Prophet, LightGBM, N-BEATS, and TFT under one consistent API, allowing classical and deep models to be benchmarked side by side with minimal extra code.

3. System Modules

1. Data Ingestion

- Connects to enterprise data sources (databases, CSVs, APIs).
- Handles multiple related series (e.g., per-product, per-region).

2. Preprocessing & Feature Engineering

- Missing value handling and outlier detection.
- Calendar features: holidays, day-of-week, promotions.
- Lag features and rolling statistics.
- Time-ordered train/validation/test splitting.

3. Modeling

- Baseline models: naive, seasonal naive, ARIMA/Prophet.
- ML models: LightGBM with engineered features.
- Optional deep learning model (N-BEATS or TFT) for multi-horizon-native forecasting.
- Model registry to compare and swap models easily.

4. Evaluation

- Metrics: MAPE, RMSE, MAE, pinball loss for probabilistic forecasts.
- Backtesting across multiple horizons.
- Business-impact metrics, such as inventory cost saved or forecast error cost.

5. Forecasting & Inference

- Generates multi-step forecasts (e.g., next 30 days).
- Optional prediction intervals for uncertainty quantification.

6. Serving & Reporting

- API endpoint for forecasts.
- Dashboard for business users.
- Alerting on anomalies or forecast drift.

7. Monitoring & Retraining

- Tracks model performance over time.
- Triggers retraining on data drift or a scheduled cadence.

4. End-to-End Workflow

- Data Ingestion — pull historical time-series data from enterprise sources.
- Exploratory Data Analysis — examine trends, seasonality, missing data, outliers, and cross-series correlations.
- Preprocessing & Feature Engineering — clean data, build lag/calendar/rolling features, split by time.
- Baseline Modeling — fit naive and statistical models (ARIMA, Prophet) as sanity-check benchmarks.
- Advanced Modeling — train ML (LightGBM) and/or DL (N-BEATS/TFT) multi-horizon models with rolling-origin cross-validation.
- Evaluation & Model Selection — compare models across horizons on MAPE/RMSE/pinball loss; balance accuracy vs. interpretability vs. compute cost.
- Business Validation — translate forecast errors into business impact and secure stakeholder sign-off.
- Deployment — wrap the selected model in a FastAPI service and connect it to a dashboard.
- Monitoring & Retraining — track live performance vs. actuals; set drift-based or scheduled retraining triggers.
- Documentation & Research Alignment — document methodology and cite relevant literature (e.g., M4/M5 competition findings).

5. Recommended Approach

Ship a strong classical/ML baseline first to deliver real-world impact quickly, then layer in one well-validated deep multi-horizon model (N-BEATS or TFT) as a documented comparison — rather than chasing the newest architecture. This keeps the system maintainable while remaining defensible in a research or technical review.